Windows / Sysmon Threat Detection System

CyberFox Society | Cybersecurity R&D Internship | Final Project Report

1. Problem & Objective

Windows and Sysmon produce large volumes of event telemetry, but event collection alone does not explain whether a pattern deserves analyst attention. This project develops an offline, rule-based minimum viable product (MVP) to flag selected suspicious Windows activity while keeping detection logic inspectable.

2. Research Context & Gap

Microsoft Sysmon records detailed host telemetry; enterprise detection platforms can correlate and investigate events but require configuration and tuning. A lightweight, transparent local prototype is useful for demonstrating event parsing, simple correlations and the limits of heuristic alerts. It is not a replacement for a SIEM or EDR.

3. System Design & Implementation

Pipeline: Windows Security/Sysmon XML export (or normalized JSON) → XML parser → normalized event records → rule-based detection → JSON alerts. Python standard library modules implement the pipeline. The XML parser handles Windows Event XML with Event/System/EventData fields, including multiple exported Event elements; it filters to Security Event ID 4625 and Sysmon Event ID 1.

4. Detection Rules

Rule A (HIGH): five or more failed logons from the same non-loopback source IP to the same target account on the same host in a ten-minute sliding window. Rule B (MEDIUM): a Sysmon process creation event for powershell.exe or pwsh.exe with selected encoded-command or hidden/no-profile command-line arguments. Alerts include rule ID, timestamp, severity, host, reason and supporting fields.

5. Scope & Ethics

This is an offline, authorized-lab prototype. No live collection, credential capture, malware execution or public release of raw event logs is required. Real event exports may contain personal and organizational data and should remain private.

Testing, Results & Next Validation

6. Completed Synthetic Tests

The existing automated suite completed 13/13 tests successfully in this package. Running the bundled synthetic XML sample produced two alerts. These checks demonstrate regression behavior for selected test fixtures only; they do not establish real-world accuracy or detection reliability.

7. Real-Log Validation Protocol

On an authorized Windows test host, enable relevant auditing and Sysmon process creation logging, generate documented positive and negative lab scenarios, export Security 4625 and Sysmon 1 events to XML, and run the detector separately for each scenario. Compare alerts against operator-recorded expected outcomes and verify event details in Event Viewer. The repository includes a PowerShell exporter, a step-by-step guide, a labeling template and a scenario-level scoring script.

8. Evaluation Measures

For each labeled scenario, compare expected versus observed rule IDs. Report true positives (TP), false positives (FP), false negatives (FN), precision = TP/(TP+FP) and recall = TP/(TP+FN) when denominators are nonzero. These are scenario-level measures, not event-level accuracy. No real-log TP/FP/FN or precision/recall is claimed because no authorized real-log export was supplied for verification.

9. Limitations & Future Work

Windows 4625 records may omit a usable source IP, preventing grouping. Legitimate PowerShell administration can match command-line heuristics. Sysmon availability and configuration vary by host. The parser reads exported XML, not native EVTX files. Next steps: run the authorized lab validation, review missed events, tune detection rules, add more event types, and compare outcomes across benign and suspicious scenarios.

10. Submission & References

Deliverables: GitHub source repository, five-slide PowerPoint, two-page report and weekly updates. References: Microsoft Sysmon documentation (learn.microsoft.com/en-us/sysinternals/downloads/sysmon); PowerShell Get-WinEvent documentation (learn.microsoft.com/en-us/powershell/module/microsoft.powershell.diagnostics/get-winevent).